

PROJET DIGICHEESE

CONCEPTION D'UNE API BACKEND



GROUPE 4 :
JULIEN MILHAU VILLAR
VICTOR ODIN
THIBAUT ODOR

LEAD DEV - 2025/2026

LE PROJET

Contexte : refonte d'un SI DigiCheese (ancienne appli Access) devenu instable et difficile à maintenir.

But : développer une API backend Python connectée à une base MySQL, testable avec swagger et pensée pour un futur front.

Approche : application centrée sur les rôles (cumulables) : Admin / Opérateur colis / Opérateur stock.

Attendus : CRUD par acteur + sécurités d'accès, + tests API (pytest) et documentation.

2025-D12 TP7 Lead Dev/DevOps EISI

Projet : Développement d'une application informatique

Objectifs finaux pour la notation du Projet :

Critères d'évaluation :

- Code source commenté et conforme aux normes
- Documentation des mécanismes de protection (standards)
- Outils et guides pour améliorer l'efficacité
- Scénarios de tests complets et détaillés, co
- Résultats de tests interprétés et correction
- Qualité du code renforcée et suivi des cor
- KPIs et alertes de monitoring définis et op
- Rapport final (assisté par IA) pertinent et a

Modalités d'évaluation :

- Livraison d'une application optimisée avec
- Rapport sur pratiques de sécurité intégrée
- Guide des outils pour l'efficacité des dével
- Documentation des mécanismes de sécuri
- accès...
- Rapport sur corrections appliquées pour a
- Monitoring avec indicateurs de performan
- Production écrite collective + restitution o

2025-D12 TP7 Lead Dev/DevOps EISI

Projet TP7 - Activité 3
appli
API BACK
Compe

- ❖ Durées du projet : 5 j
- ❖ Livrables par un lien G
- ❖ Une branche par
- ❖ test, dev, prod et
- ❖ Sources python p
- ❖ Mysql)
- ❖ Un dossier avec
- (Dans un README.md
- ❖ Un dossier conten
- ❖ Un dossier conten
- (contenant l'architectu
- charges fournis, la des
- ❖ [Faire un requiem](#)
- ❖ [Les consignes : co](#)

Diginamic - Fiche TP7 Développement d'...
le 01/2025 - CG

Mini Cahier des charges fonctionnels

Refonte SI de la gestion des cadeaux
fidélité Fromagerie DIGICHEES

ORGANISATION

RÉPARTITION



Julien

Routage



Victor

Conception / Test



Thibault

Modélisation

Tous

GitHub (branches/PR), relecture, intégration,
amélioration, refactorisation

ORGANISATION

Daily meeting pour: Code review, Merge Pull Request,
Tâches en cours / A faire

Mercredi 14	Jeudi 15	Vendredi 16	Dimanche 18	Lundi 19	Mardi 20
- Initialisation du projet - Mise en place de la structure Flask - Premières routes de test (création et authentification).	- Modélisation des données - Gestion authentification et rôles utilisateurs - Mise en place de Swagger.	- Repositories - Implémentation des repositories - Refactorisation Backend	- Refactorisation - Absraction BaseRepository - Implémentation tests - Stabilisation API	- Présentation soutenance - Mise en place tests	- Finalisation des rendus - Présentation soutenance

OUTILS UTILISÉS

Backend / API

Python + Flask : base de l'application

Flask-RESTful : endpoints REST

Flask-SQLAlchemy / SQLAlchemy : ORM + accès BDD

PyMySQL + MySQL : connexion à la base

Sécurité & Auth

Flask-Login : gestion de session / login

Werkzeug security : hash de mot de passe

Doc & Qualité

Flasgger (Swagger UI) : documentation interactive (YAML dans les docstrings)

pytest + pytest-flask : tests unitaires / tests endpoints

Modélisation

Draw.io

Collaboration

Git / GitHub : branches, pull requests, merges (workflow d'équipe)

Aide

Chat GPT

POURQUOI CES OUTILS

Contrainte Techniques:

- Swagger
- Python
- SQLAlchemy
- pytest

Flask:

- Gestion des sessions
- Similitude avec FastAPI (Framework léger)
- Mise en place rapide
- Intégration naturelle avec les contraintes techniques (Flask-SQLAlchemy, Flask, PyMySQL, Pytest-Flask)

Collaboration (Github):

- Gestions multibranches pour développements parallèles
- Historisation et versionnage
- Code review

ARCHITECTURE

Routes (Blueprints)

Exposent les endpoints REST (HTTP), contrôles d'accès (rôles), réponse JSON

Repositories

Centralisent le CRUD et l'accès BDD
(moins de duplication, code plus testable)

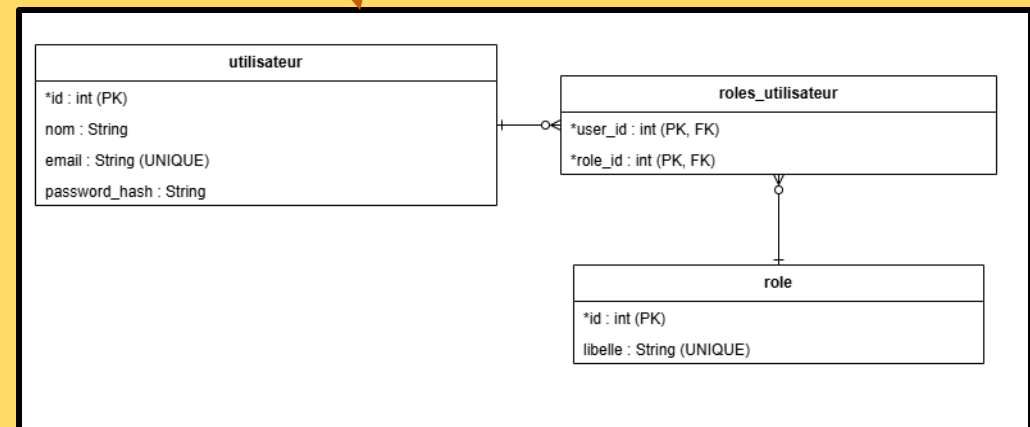
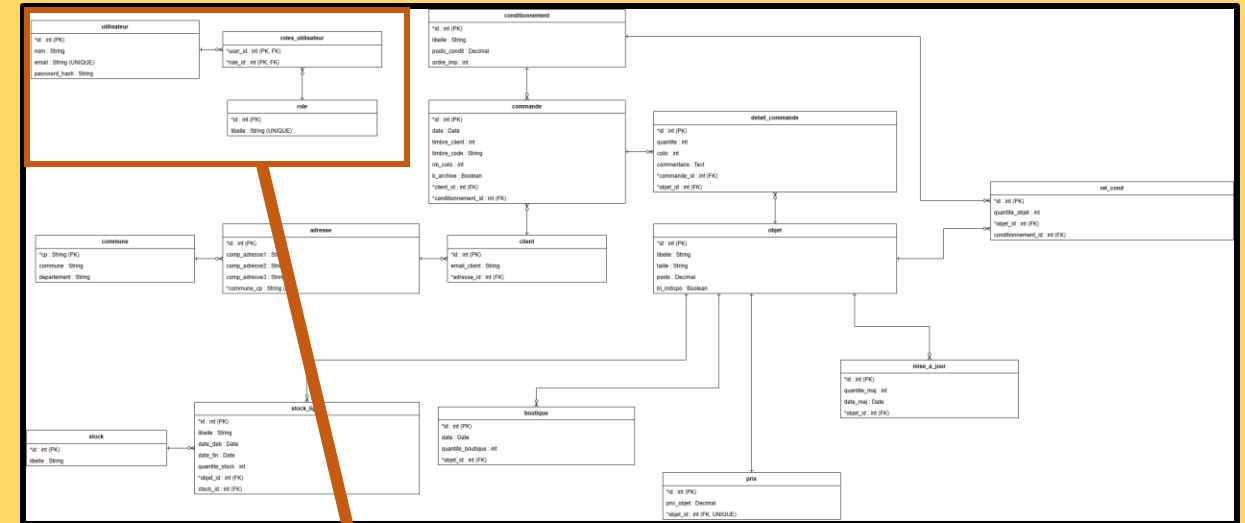
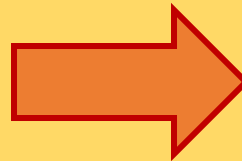
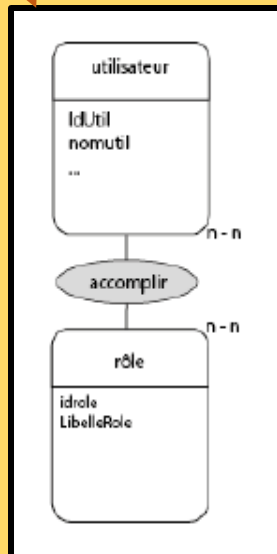
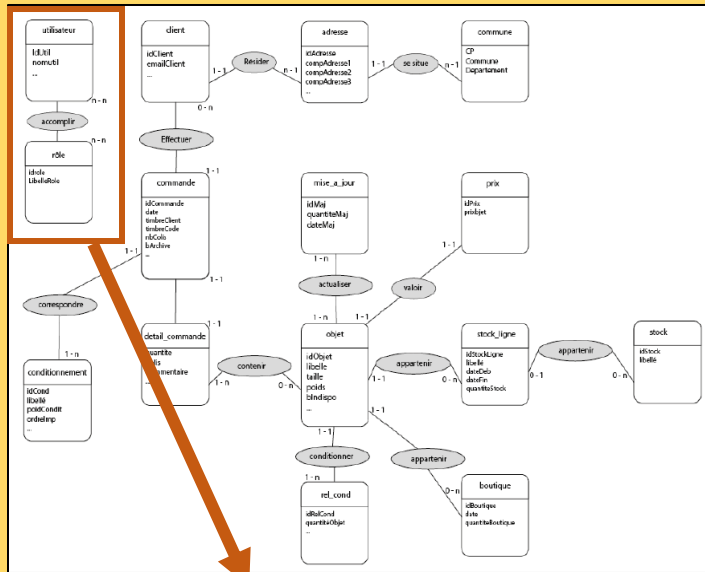
Models (SQLAlchemy)

Définissent les tables, relations et la structure des données (ORM)

Tests (Pytest-flask)

Robustesse et prédictibilité du code, évite les effets de bords

MODÈLE DE DONNÉES



CONCEPTION DES ROUTES

Décorateur de rôle

Structure Initiale

- Routes Admin
- Routes Operateur Colis



Structure Finale

Authentification

- Login
- Signup

Admin

- Utilisateurs
- Conditionnements
- Communes
- Objets / Goodies

Operateur Colis

- Adresses
- Clients
- Commandes
- Détail Commandes

Route en développement

- Mailer
- Statistiques

DÉMO API

Présentation des routes

<http://127.0.0.1:5000/apidocs/>

Authentication

<http://127.0.0.1:5000/api-login>

<http://127.0.0.1:5000/api-logout>

CRUD ADMIN

<http://127.0.0.1:5000/admin/users>

CRUD OPERATEUR COLIS

<http://127.0.0.1:5000/colis/clients>

Routes protégées

<http://127.0.0.1:5000/colis/clients>

TESTS

Structure:

```
--Tests  
-----conftest.py  
-----test_admin.py  
-----test_auth.py  
-----test_colis.py
```



Conftest (Setup):

- Création du client test
- Création des fixtures (data)
- Création des fixtures (fonctionnelles)

CONCLUSION

Futures améliorations

- Documentations complètes
- Services
- Mailler / Statistiques
- Etendre l'existant pour le rôle operateur stock
- Testing étendu
- Réfléchir à de potentiels améliorations de la modélisation